RESULTS

Exam Results

826 / 1000

PASSED

50 of 60 correct (83%) · Pass mark: 720

Domain Breakdown

D1	Agentic Architecture & Orchestration	11/14 (79%)	+212pts
D2	Tool Design & MCP Integration	8/11 (73%)	+131pts
D3	Claude Code Configuration & Workflows	9/12 (75%)	+150pts
D4	Prompt Engineering & Structured Output	11/12 (92%)	+183pts
D5	Context Management & Reliability	11/11 (100%)	+150pts

HOW SCORING WORKS

Each domain's percentage correct is multiplied by its weight and scaled to 1000 points. For example, scoring 80% in Agentic Architecture (27% weight) contributes 0.80 × 0.27 × 1000 = 216 points. The pass mark is 720/1000 (72%). If a domain has no questions in the selected scenarios, its weight is redistributed proportionally among assessed domains.

The real exam reports a scaled score from 100 to 1,000 plus your percent-correct by domain, so treat the percentages here as a readiness gauge rather than a prediction of your scaled score.

HIDE DETAIL TAKE ANOTHER EXAM

Question-by-Question Review

ALL 60 CORRECT 50 INCORRECT 10 SKIPPED 0

Q01 · D5 5.5 HUMAN-REVIEW-CALIBRATION / REVIEWER-CAPACITY q-5-5-007 ·
Technical Documentation Maintenance System CORRECT

Claude Code generates API reference pages for 150 endpoints across three categories: payment processing (30 endpoints, high regulatory risk), internal tooling (80 endpoints, low risk), and public data queries (40 endpoints, moderate risk). The team cannot manually review all 150. How should they structure human review?

A Randomly sample 15% of all endpoints (approximately 23 pages) for review, ensuring a representative cross-section

Uniform random sampling treats all endpoints equally, but payment processing endpoints carry much higher risk. A 15% sample might only include 4-5 payment endpoints, inadequately covering the highest-risk category.

B Use stratified sampling: review 100% of payment docs, 10% of public data query docs, and 5% of internal tooling docs.

Stratified sampling allocates review effort proportional to risk. Payment processing documentation has regulatory implications warranting full review. Public data queries affect external users and deserve moderate scrutiny. Internal tooling has the lowest blast radius and can tolerate the lightest review. This achieves thorough coverage of critical content within a manageable review budget.

D Have Claude Code self-assess each generated page with a confidence score and only review pages where confidence falls below 80%

LLM self-reported confidence is poorly calibrated. Claude Code may report high confidence on pages with subtle factual errors (e.g., incorrect parameter types, wrong default values) because the generated text is fluent and internally consistent.

C Review only the payment processing endpoints since they are the highest risk, and trust Claude Code's output for the remaining 120 endpoints

Completely skipping review for 120 endpoints risks undetected errors in public-facing documentation. Even low-risk internal tooling docs should have some sample review to catch systematic generation issues that might affect all

categories.

WHERE THIS COMES FROM

Lesson 5.5: Human Review and Confidence Calibration (Reviewer capacity)

Lesson 5.5: Human Review and Confidence Calibration (Stratified sampling)

Q02 · D4 4.6 MULTI-PASS-REVIEW / MULTI-PASS q-4-6-004 · CI/CD Pipeline Integration CORRECT

Your CI/CD code review system analyses pull requests averaging 8-12 files. Reviews are thorough on the first few files but become increasingly superficial on later files, sometimes missing obvious issues. What architectural change best addresses this pattern?

B Increase the model's context window to give it more space to analyse all files

Attention dilution is not a context window size problem. The model has enough space but distributes attention unevenly across many files.

C Split the review into per-file passes so each file gets dedicated attention, then a cross-file integration pass.

Per-file analysis ensures each file receives consistent, thorough attention. The cross-file integration pass then catches issues that span multiple files, such as inconsistent interfaces or data flow problems. This directly solves attention dilution.

D Add a second full-PR review pass and merge findings from both passes

A second pass on the full PR suffers from the same attention dilution. Both passes will likely be thorough on early files and superficial on later ones.

A Randomise the file order before each review run so a different subset of files receives the thorough early-pass attention each time.

Randomising order means different files get superficial treatment each time, but does not ensure all files receive thorough review. The root cause — attention dilution — remains.

WHERE THIS COMES FROM

Lesson 4.6: Multi-Instance Review and Output Validation (Multi-pass review)

Lesson 4.6: Multi-Instance Review and Output Validation (Why context windows do not fix it)

Q03 · D4 4.1 SYSTEM-PROMPTS / FALSE-POSITIVES q-4-1-001 · CI/CD Pipeline Integration INCORRECT

Your CI/CD code review pipeline has a 40% false positive rate on 'documentation mismatch' findings, causing developers to ignore ALL review categories. What is the most effective fix?

A Add 'only report high-confidence documentation issues' to the system prompt

Vague confidence instructions do not improve precision. The model has no concrete criteria for what 'high-confidence' means.

B Temporarily disable the documentation mismatch category while refining its prompts with explicit criteria and code examples

This restores trust in other categories immediately while you iterate on the problematic category with specific criteria. (correct answer)

D Add a second model pass to verify each finding before reporting

A second pass without better criteria will have the same false positive problem. Fix the criteria first. (your answer)

C Increase the model temperature to get more varied results and filter outliers

Temperature affects randomness, not precision. This would likely increase false positives.

WHERE THIS COMES FROM

Lesson 4.1: System Prompts with Explicit Criteria (False-positive trust problem)

Q04 · D2 2.2 STRUCTURED-ERROR-RESPONSES / ERROR-CATEGORIES q-2-2-007 · Enterprise Data Platform with Federated Queries CORRECT

A `query_postgres` MCP tool returns 500 on transient server overload and 404 on missing tables. The agent retries both identically, exhausting retries on missing tables and then failing to retry once the server recovers. What change to the error response structure would fix this?

C Remove the retry logic entirely and escalate all errors to the user immediately for manual resolution.

Removing retries eliminates the agent's ability to recover from transient failures automatically. The goal is smarter retries, not no retries.

D Increase the retry limit from 3 to 10 so the agent has enough attempts to cover both transient and permanent errors.

More retries waste time on permanent errors (missing tables will never appear) and add unnecessary latency. The problem is retry categorisation, not retry count.

B Add a structured category field so the agent retries transient errors but not permanent errors like a missing table.

Structured error categories give the agent explicit recovery guidance. Transient errors (overload, timeout) warrant retries; a missing table is a permanent failure that retrying never resolves, since the table still will not exist on the next attempt. This eliminates wasted retries on unrecoverable errors.

A Return all errors with the MCP isError flag set to true and include the HTTP status code in the error message text so the agent can parse it.

Embedding status codes in free-text messages requires the agent to parse unstructured text, which is unreliable. Structured error categorisation is more robust than parsing status codes from strings.

WHERE THIS COMES FROM

Lesson 2.2: Structured Error Responses (Four error categories)

MCP: Tools Specification ↗

Q05 · D5 5.4 CODEBASE-EXPLORATION / CONTEXT-DEGRADATION q-5-4-008 ·

Technical Documentation Maintenance System CORRECT

A documentation team working across a 500,000-line codebase notices that Claude Code's responses become less specific after extensive file exploration. What is the term for this degradation and what is the primary mitigation in Claude Code?

A Token exhaustion, mitigated by raising max_tokens so the model has room to keep referencing every file it has read.

max_tokens controls response length, not the model's ability to retain and reference earlier context. The degradation occurs because verbose exploration output fills the context window, not because responses are truncated.

C Hallucination drift — mitigated by lowering the temperature to zero for deterministic output

The issue is not hallucination from randomness but loss of specificity from context overload. Temperature zero produces deterministic output but does not help the model retain specific findings from earlier in a long context.

D Prompt injection — mitigated by sanitising file contents before they enter the context window

Prompt injection is a security concern involving malicious content in inputs. The described problem — responses becoming vague after extensive exploration — is context degradation from accumulated verbose output, not a security issue.

B Context degradation, mitigated by scratchpad files and by delegating verbose exploration to subagents.

Context degradation occurs when the model loses grip on specific details as verbose tool output accumulates in the conversation. Scratchpad files persist findings on disk where they can be re-read as needed. Subagent delegation isolates verbose exploration output from the main agent's context budget.

WHERE THIS COMES FROM

Lesson 5.4: Codebase Exploration and Context Degradation (Context degradation)

Lesson 5.4: Codebase Exploration and Context Degradation (Scratchpad mitigation)

Q06 · D3 3.4 PLAN-MODE-EXECUTION / EXPLORE-SUBAGENT q-3-4-007 ·

Large-Scale Codebase Refactoring with Multi-Agent Claude Code INCORRECT

A developer asks Claude Code to 'explain the order processing pipeline' before refactoring it. The pipeline spans 15 classes across 4 packages with deep inheritance and callback chains. Claude Code reads 3 files and produces a superficial summary that misses the callback chain entirely. What approach would produce a more thorough analysis?

C Switch to plan mode and ask Claude Code to create a plan for understanding the pipeline

Plan mode can safely explore a codebase before you commit to changes, but it runs in the main conversation and drives toward a change plan. Here the developer needs thorough, verbose discovery of current behaviour, which the Explore subagent isolates and summarises without exhausting the main context, so the Explore subagent is the better fit. (your answer)

D Run multiple Claude Code sessions, each analysing a different package, then manually combine the results

Splitting analysis across sessions loses the cross-package context needed to trace the callback chains and inheritance hierarchies. The pipeline's complexity comes from how the 4 packages interact, which requires a single comprehensive analysis.

A Re-prompt Claude Code with more specific instructions listing all 15 classes to read

This assumes the developer already knows which 15 classes are involved. If the developer knew the full scope, they would not need Claude Code's help understanding the pipeline. The issue is that Claude Code's initial exploration was too shallow.

B Use an Explore subagent for verbose discovery across the pipeline's classes and callback chains

The Explore subagent is purpose-built for verbose, thorough codebase discovery. It systematically follows references, traces inheritance, and maps control flow across files — exactly what is needed for a complex event-driven pipeline. Regular Claude Code defaults to efficient, minimal reads; the Explore subagent prioritises comprehensive discovery. (correct answer)

WHERE THIS COMES FROM

Lesson 3.4: Plan Mode vs Direct Execution (Explore subagent)

Q07 · D1 1.1 AGENTIC-LOOPS / TOOL-RESULT-HANDLING q-1-1-009 CORRECT

A customer-support agent runs an agentic loop. Each turn it inspects `stop_reason`; when the value is `"tool_use"` it executes the requested tool and receives the tool's output. Before it calls the model again so the loop can continue coherently, what must the agent do with that output?

B Replace the previous assistant message with the tool output to keep the context window small

Overwriting the assistant message discards the tool_use block that the result corresponds to, breaking the tool_use/tool_result pairing the API requires and losing the reasoning that led to the call.

A Append the tool result to the conversation history as a new message, then send the full updated conversation on the next call.

The Messages API is stateless, so every call must carry the whole conversation. The tool result is appended as a new message paired with the tool_use block that requested it, which is how the model sees what its call returned and decides the next step.

D Store the tool output in an external store and pass a reference id to the model on the next call

The model cannot dereference an external id. The tool result has to be present in the conversation the model actually receives.

C Send only the tool output on the next call, since the API retains the earlier turn server-side and will pair it with the pending tool request.

The Messages API keeps no server-side memory of the turn. Sending only the tool output drops all prior context, so the model cannot continue the task coherently.

WHERE THIS COMES FROM

Lesson 1.1: Agentic Loops (Tool result handling)

Anthropic: Claude Agent SDK Overview ↗

Q08 · D3 3.1 CLAUDE-MD-HIERARCHY / ENFORCEMENT q-3-1-007 ·

Large-Scale Codebase Refactoring with Multi-Agent Claude Code CORRECT

Project-level `.claude/CLAUDE.md` says 'use 4-space indentation matching the existing codebase.' A senior architect has 'use 2-space indentation' in their user-level `~/.claude/CLAUDE.md`. In recent sessions the architect's code has come back in 2 spaces and broken the build. The team needs a guarantee that 4-space indentation is applied on every save. What should they do?

C Ask the architect to delete their user-level `~/.claude/CLAUDE.md` so there is no conflict to resolve

This treats a personal config file as a team problem and doesn't scale — every new teammate would need to police their own home directory, and any future contradiction (from a different file, an `@import`, or a `.claude/rules/` entry) would resurface the same fragility. The fix is to remove reliance on guidance-style config for a rule that must be enforced, not to remove the conflicting file.

A Add a PostToolUse hook that runs the team's formatter after every Write/Edit, so 4-space indentation is enforced regardless of what Claude generates

Anthropic's memory docs are explicit that CLAUDE.md is delivered as a user message with 'no guarantee of strict compliance,' and that 'if two rules contradict each other, Claude may pick one arbitrarily.' The docs themselves point at hooks for this case: hooks 'execute as shell commands at fixed lifecycle events and apply regardless of what Claude decides to do.' A PostToolUse hook running the formatter is the only option here that gives a hard guarantee.

B Leave the rule in project-level `.claude/CLAUDE.md` — the more specific scope wins on conflicts, so the project rule will override the architect's user-level preference

This is the popular paraphrase, but it's not what Anthropic's docs say. The docs describe a load order (broadest scope to most specific, so project instructions appear in context after user instructions) but explicitly state files are 'concatenated into context rather than overriding each other' and conflicts 'may [be] pick[ed] arbitrarily.' The team has already seen that assumption fail in production.

D Move the 4-space rule into a `CLAUDE.local.md` at the project root so it is appended last and reads after the user-level file

Load order does append `CLAUDE.local.md` after `CLAUDE.md` within a directory, but the docs are careful to call this 'load order,' not precedence, and warn that conflicts may resolve arbitrarily. `CLAUDE.local.md` is also gitignored, so a team standard cannot live there.

WHERE THIS COMES FROM

Lesson 3.1: CLAUDE.md Hierarchy and Scoping (Loading order and conflict handling)

Anthropic — How Claude remembers your project (memory docs) ↗

Q09 · D1 1.6 TASK-DECOMPOSITION / ADAPTIVE-DECOMPOSITION q-1-6-009 ·

Large-Scale Codebase Refactoring with Multi-Agent Claude Code INCORRECT

You are planning how Claude should tackle a large, unfamiliar refactoring effort. Which characteristics of the work indicate dynamic adaptive decomposition rather than a fixed sequential pipeline? (Select 2)

A The review covers the same predictable set of aspects on every run.

A predictable multi-aspect review is exactly where a fixed prompt-chaining pipeline fits best.

D The task is open-ended, like adding comprehensive tests to a legacy codebase you have not mapped yet.

This is the guide's example of open-ended work: map the structure first, then build a prioritised plan that adapts as dependencies surface.

C Each step's output feeds the next in a stable, known order.

A stable, known step order is the defining property of a sequential pipeline, not of adaptive decomposition.

B The useful subtasks only become clear as intermediate findings come in.

Adaptive plans generate subtasks from what each step discovers, which a fixed pipeline cannot do. (correct answer)

WHERE THIS COMES FROM

Lesson 1.6: Task Decomposition Strategies (Dynamic adaptive decomposition)

Lesson 1.6: Task Decomposition Strategies (Sequential pipelines)

Q10 · D1 1.4 WORKFLOW-ENFORCEMENT-HANDOFF / PREREQUISITE-GATES q-1-4-012 ·

Large-Scale Codebase Refactoring with Multi-Agent Claude Code CORRECT

The refactoring coordinator delegates a database migration task to a subagent. The subagent sometimes executes destructive SQL migrations (DROP TABLE, ALTER TABLE DROP COLUMN) before verifying that a rollback script exists. The team requires that every destructive migration has a verified rollback before execution. What approach provides the strongest guarantee?

B A programmatic prerequisite gate that blocks any DROP or ALTER TABLE DROP until a rollback verification returns success.

A programmatic prerequisite gate physically prevents destructive SQL execution until the rollback verification precondition is met. This provides a deterministic guarantee that no destructive migration runs without a verified rollback, eliminating the failure mode entirely.

C Have the coordinator agent review the subagent's SQL before allowing execution

The coordinator is another LLM agent and provides probabilistic review, not deterministic enforcement. It may also miss destructive patterns in complex SQL statements. Programmatic gates are the correct approach for safety-critical operations.

D Run all migrations in a sandboxed database first and check for errors before applying to the real database

Sandboxing tests for execution errors, not for the existence of rollback scripts. A migration that runs successfully in a sandbox still lacks a rollback plan. The requirement is specifically about verifying rollback scripts before execution.

A Add a prerequisite check in the system prompt: 'Always verify rollback scripts exist before executing destructive migrations'

Prompt instructions are probabilistic. The agent 'sometimes' skips verification, proving that instructions alone are insufficient. Destructive database operations require deterministic guardrails.

WHERE THIS COMES FROM

Lesson 1.4: Workflow Enforcement and Handoff (Prerequisite gates)

Lesson 1.5: Agent SDK Hooks (PreToolUse hooks)

Q11 · D4 4.6 MULTI-PASS-REVIEW / SELF-REVIEW-BIAS q-4-6-002 · CI/CD Pipeline Integration CORRECT

Your Claude Code setup reviews generated code by asking the same model instance to critique its own output immediately after generation, within the same conversation. The team notices the reviews are overly favourable and miss bugs that an external reviewer would catch. What is the root cause and correct fix?

D Lower the temperature during the review phase to make the model more precise and less likely to approve questionable code

Temperature affects response variability, not the fundamental bias of self-review. The model will still be biased towards its own output regardless of temperature settings. An independent instance is the correct architectural fix.

A The model needs more detailed review instructions; add a comprehensive checklist of what to look for during self-review

More detailed instructions do not overcome the fundamental limitation. The model retains its reasoning context from generation and is less likely to question its own decisions regardless of how detailed the checklist is.

B Enable extended thinking so the model reasons more carefully about potential issues in its own code

Extended thinking does not solve the self-review limitation. The model still retains its reasoning context and is biased towards its own output. Extended thinking makes the existing reasoning deeper, not more independent.

C Route the review to a separate Claude instance that has no access to the generation conversation

A model reviewing its own output in the same session retains reasoning context and is less likely to question its own decisions. An independent instance without prior context catches more subtle issues because it approaches the code fresh.

WHERE THIS COMES FROM

Lesson 4.6: Multi-Instance Review and Output Validation (Self-review limitation)

Q12 · D5 5.1 CONTEXT-WINDOW-MANAGEMENT / LOST-IN-THE-MIDDLE q-5-1-008 · Technical Documentation Maintenance System CORRECT

Claude Code is synthesising release notes from commit messages, PR descriptions, and changelog entries. During a 200+ commit session, summaries of early commits become vague ('various bug fixes') while recent commits are still detailed accurately. The context window is not full. What is the most likely cause?

A The model's temperature is set too high, causing it to generate vague summaries randomly

Temperature affects output randomness but would produce inconsistent quality across all commits, not a systematic pattern where early commits are vague and recent commits are accurate.

D Claude Code applies progressive summarisation to older commits to conserve context for recent ones

Claude Code does not automatically apply progressive summarisation to tool results within a single processing step. The context window is not full, so there is no space pressure triggering summarisation. The issue is attention distribution across long inputs.

C The commit messages for early commits are inherently less detailed than recent ones, so the vague summaries are accurate

The question states the vagueness applies to early commits specifically, not commits with poor messages. The systematic pattern (early = vague, recent = detailed) points to a context positioning effect, not source quality variation.

B The lost-in-the-middle effect: the model favours the start and end of long input, losing middle commit detail.

The lost-in-the-middle effect is a well-documented phenomenon where models attend more strongly to the start and end of long contexts, with reduced attention to middle content. With 200+ commits loaded sequentially, commits in the middle of the sequence receive less attention, producing vague summaries. Processing in smaller batches or reordering critical information to the start and end mitigates this.

WHERE THIS COMES FROM

Lesson 5.1: Context Window Management (Lost in the middle)

Q13 · D3 3.6 CICD-INTEGRATION / OUTPUT-FORMAT q-3-6-004 · CI/CD Pipeline Integration CORRECT

A team wants their CI pipeline to run Claude Code for both PR code review and automated test generation. The PR review should output structured JSON for integration with their review dashboard, while the test generation should produce standard text output. How should the two pipeline steps be configured?

D Run both steps with --output-format json and have the test generation step extract code from the JSON response

While this would work technically, it adds unnecessary complexity to the test generation step. Using JSON output format only for the step that needs structured parsing (review) is the cleaner approach.

C Run both steps in a single Claude Code session with -p, using different prompts to request different output formats

Running both steps in a single session violates session context isolation. The review step would retain reasoning context from test generation, reducing review effectiveness. Each step should be an independent invocation.

A Run both steps with -p and configure the review dashboard to parse plain text output from both steps

Plain text output requires fragile parsing logic in the review dashboard. The --output-format json flag provides structured, machine-readable output that is far more reliable for programmatic integration.

B Run the review step with -p --output-format json and the test generation step with -p only, as separate non-interactive invocations

Each step uses -p for non-interactive mode. The review step adds --output-format json for structured output that the dashboard can parse reliably. The test generation step uses default text output since it produces code files, not structured data. Running them as separate invocations ensures session context isolation.

WHERE THIS COMES FROM

Lesson 3.6: CI/CD Integration (Structured output)

Lesson 3.6: CI/CD Integration (-p flag)

Claude Code: Headless / CLI Reference ↗

Q14 · D4 4.6 MULTI-PASS-REVIEW / MULTI-PASS q-4-6-001 · CI/CD Pipeline Integration CORRECT

A PR modifying 14 files receives inconsistent review: detailed feedback on some files, superficial comments on others, and contradictory findings (flagging a pattern in one file while approving identical code elsewhere). What is the best restructuring?

B Run three independent passes on the full PR and only flag issues found in at least two passes

This suppresses real bug detection by requiring consensus on issues that may only be caught intermittently.

C Split into per-file local analysis passes plus a separate cross-file integration pass

Per-file analysis ensures consistent depth. The integration pass catches cross-file data flow issues. This directly addresses attention dilution.

D Require developers to split large PRs into smaller submissions of 3-4 files

This shifts burden to developers without improving the review system itself.

A Switch to a higher-tier model with a larger context window

Larger context windows do not solve attention quality issues. The problem is attention dilution, not context size.

WHERE THIS COMES FROM

Lesson 4.6: Multi-Instance Review and Output Validation (Multi-pass review)

Q15 · D5 5.1 CONTEXT-WINDOW-MANAGEMENT / LOST-IN-THE-MIDDLE q-5-1-007 · Enterprise Data Platform with Federated Queries CORRECT

An agent receives 200 rows of Snowflake financial data and places it mid-prompt, between system instructions and the user's follow-up question. Asked to identify the row with the highest margin, the agent picks the 45th row (margin 32%) over the 142nd row (margin 47%). What cognitive bias is affecting the agent?

A Recency bias — the agent prioritises data appearing near the end of the context window.

Recency bias would favour rows near the end of the data, not the 45th row. The 45th row is in the early portion of the result set, which is consistent with the lost-in-the-middle effect, not recency bias.

D Anchoring bias — the agent fixates on the first high-margin row it encounters and stops scanning the rest.

Anchoring bias is a human cognitive bias, not a well-documented LLM failure mode for structured data scanning. The observed behaviour — accurate recall at the start and end, poor recall in the middle — is the hallmark of the lost-in-the-middle effect.

B The lost-in-the-middle effect: the agent favours the start and end, missing the row buried in the middle.

The lost-in-the-middle effect causes LLMs to attend disproportionately to content at the start and end of long sequences, with reduced attention to material in the middle. Row 142 (the correct answer) sits deep in the middle of the 200-row result set, making it prone to being overlooked.

C Token limit truncation — the result set exceeds the context window and rows beyond the 100th are silently dropped.

If rows were truncated, the agent would not have access to row 142 at all and would report the highest margin from the visible rows. The agent identified a specific wrong row (45th), indicating all data is present but attention is uneven.

WHERE THIS COMES FROM

Lesson 5.1: Context Window Management (Lost in the middle)

Q16 · D2 2.4 MCP-SERVER-INTEGRATION / CONFIG-SCOPE q-2-4-008 · Enterprise Data Platform with Federated Queries CORRECT

The data platform team needs to share their Snowflake and PostgreSQL MCP server configurations with all team members whilst allowing individual developers to experiment with a personal API integration server. Where should each configuration be placed?

D Place the database servers in environment variables and reference them in both .mcp.json and ~/.claude.json.

Environment variables are used for credentials (connection strings, API keys), not for MCP server configuration. The server definitions themselves belong in .mcp.json or ~/.claude.json depending on scope.

B Place the Snowflake and PostgreSQL servers in the project-level .mcp.json, and personal API integration servers in user-level ~/.claude.json.

Project-level .mcp.json is version-controlled and shared — correct for team-wide database integrations. User-level ~/.claude.json is machine-specific — correct for personal experimental servers that should not affect other team members.

C Place all three servers in ~/.claude.json and have each developer copy the configuration manually.

User-level configuration is not version-controlled and requires manual synchronisation. Team-wide servers belong in project-level .mcp.json for consistency and change tracking.

A Place all three servers in the project-level .mcp.json so they are version-controlled and consistent across the team.

Personal experimental servers should not be in project-level configuration. Changes to personal servers would create unnecessary merge conflicts and force experimental integrations on the entire team.

WHERE THIS COMES FROM

Lesson 2.4: MCP Server Integration (Scoping hierarchy)

Claude Code: MCP Server Configuration ↗

Q17 · D2 2.4 MCP-SERVER-INTEGRATION / DESCRIPTIONS q-2-4-006 · Enterprise Data Platform with Federated Queries

INCORRECT

An MCP server exposes a `query_database` tool for Snowflake. Agents ignore it and run SQL via the built-in Bash tool against the Snowflake CLI, even though the MCP tool returns structured results with column types and pagination that Bash does not. What is the most likely cause and fix?

D Add a system prompt instruction telling the agent to always use query_database instead of Bash for SQL queries.

Whilst a system prompt instruction could work as a short-term fix, it is brittle and does not scale. The root cause is the sparse description. Enhancing the description is the sustainable fix that works across all agents and contexts. (your answer)

B Enhance the sparse description to spell out the tool's structured output and pagination advantages over Bash.

When MCP tool descriptions are sparse, agents default to the familiar Bash tool. Enhancing the description to explain the structured results, column types, and pagination gives the agent enough to prefer the MCP tool for database queries. (correct answer)

A The MCP server is not properly connected. Restart the MCP server and verify the connection with a test query.

If the server were disconnected, the tool would not appear in the available tools list at all. The agent is choosing not to use it, which is a description problem, not a connectivity problem.

C Disable the Bash tool entirely so the agent is forced to use the MCP tool for all operations.

Disabling Bash removes a critical general-purpose tool needed for many operations beyond database queries. The fix should make the MCP tool more attractive for its specific use case, not remove other tools.

WHERE THIS COMES FROM

Lesson 2.4: MCP Server Integration (Enhancing MCP descriptions)

Q18 · D3 3.6 CICD-INTEGRATION / WORKTREE-PARALLEL q-3-6-008 ·

Large-Scale Codebase Refactoring with Multi-Agent Claude Code

CORRECT

The team wants three Claude Code instances working in parallel: one extracting the authentication service, one extracting the billing service, and one updating shared libraries. All three need to commit to the same repository without conflicts. What is the correct setup?

D Use fork_session to run the three tasks as parallel branches within a single Claude Code session

fork_session creates parallel exploration branches within Claude Code's conversation context, not separate file system environments. Three large extraction tasks each need their own working directory to avoid file conflicts, which requires git worktree.

B Use git worktree to create three separate working directories, each on its own branch, with one Claude Code instance per worktree

git worktree creates multiple working directories from the same repository, each checked out to a different branch. Each Claude Code instance operates in its own isolated file system while sharing the same git history. This eliminates file system conflicts between parallel instances.

C Clone the repository three times into separate directories and merge the results manually

Three separate clones create divergent git histories that must be reconciled manually. git worktree provides the same isolation with a shared repository, making merges straightforward through normal branch operations.

A Run all three instances in the same working directory on separate branches, switching branches as needed

Multiple Claude Code instances in the same working directory will cause file system conflicts, dirty working trees, and race conditions, even on different branches. Git branch switching affects the entire working directory.

WHERE THIS COMES FROM

Anthropic: Claude Code Documentation ↗

Git: Worktrees ↗

Q19 · D5 5.4 CODEBASE-EXPLORATION / SUBAGENT-DELEGATION q-5-4-005 ·

Technical Documentation Maintenance System CORRECT

Claude Code is auditing API documentation coverage across a large codebase. After exploring 15 modules, the agent produces vague references like 'the authentication module follows standard patterns' instead of citing specific class names and method signatures it discovered earlier. `/compact` has already been used once. What is the most effective next step?

C Start a fresh session and re-explore all 15 modules more efficiently to rebuild the context with only essential findings.

Re-exploring 15 modules wastes significant time and effort. The specific findings from earlier exploration are lost. A scratchpad file would have preserved them, and subagent delegation prevents the problem going forward.

A Run /compact again to further reduce context usage and continue the exploration in the same session.

Running /compact again may help marginally, but the fundamental problem is that the agent's context has filled with verbose discovery output from 15 modules. Compacting alone does not restore the specific findings that have been lost to context degradation.

B Delegate remaining module explorations to subagents, giving each a scratchpad summary of findings so far.

Subagent delegation isolates verbose exploration output from the main agent's context. The scratchpad file persists specific findings (class names, method signatures) across context boundaries. The main agent retains enough context for coordination without being overwhelmed by discovery output.

D Increase the model's temperature to encourage more detailed and specific outputs instead of vague pattern references.

Temperature affects output randomness, not the model's ability to recall specific findings from earlier in a long context. The problem is context degradation from verbose output accumulation, not generation settings.

WHERE THIS COMES FROM

Lesson 5.4: Codebase Exploration and Context Degradation (Subagent delegation)

Lesson 5.4: Codebase Exploration and Context Degradation (Scratchpad files)

Q20 · D1 1.5 AGENT-SDK-HOOKS / POSTTOOLUSE-NORMALISATION q-1-5-007 ·

Large-Scale Codebase Refactoring with Multi-Agent Claude Code INCORRECT

During the monolith-to-microservices refactoring, each extracted service must follow a consistent Java package naming convention (com.company.service.<service-name>). The team notices that subagents sometimes use inconsistent package names like com.company.app.<service-name> or com.company.<service-name>. A team member proposes adding stronger instructions to the system prompt. What is the correct approach?

C Create a separate validation subagent that reviews all written files for naming compliance after each refactoring batch

A validation subagent is still probabilistic — it may miss violations. PostToolUse hooks provide deterministic, inline enforcement that catches every violation at the point of file creation.

D Use tool_choice to restrict the agent to a custom write_java_file tool that enforces the naming convention in its implementation

Creating a custom tool wrapper adds unnecessary complexity. PostToolUse hooks on the existing file write tool achieve the same deterministic enforcement without modifying the tool interface. (your answer)

A Add the naming convention to the system prompt with three concrete examples showing the correct pattern

Prompt instructions with examples improve consistency but still have a non-zero failure rate. Package naming is a deterministic rule that must be enforced 100% of the time to avoid broken imports across services.

B Implement a PostToolUse hook that inspects written files and normalises any package declarations to the correct com.company.service.<service-name> format

A PostToolUse hook on file write operations can inspect the output and normalise package names deterministically. This provides a 100% enforcement guarantee regardless of what the model generates, which is the correct approach for structural consistency rules. (correct answer)

WHERE THIS COMES FROM

Lesson 1.5: Agent SDK Hooks (PostToolUse normalisation)

Anthropic: Agent SDK Hooks ↗

Q21 · D1 1.6 TASK-DECOMPOSITION / DELEGATION q-1-6-008 ·

Large-Scale Codebase Refactoring with Multi-Agent Claude Code CORRECT

A coordinator agent receives a request to extract a payment processing module from the monolith into a standalone microservice. The task involves creating new API endpoints, migrating database tables, updating 40+ call sites, and writing integration tests. A junior developer suggests the coordinator should handle the entire task itself to avoid subagent communication overhead. Why is this approach wrong?

A The coordinator should never write code — it should only route tasks

The coordinator can handle simple, well-scoped tasks directly. The issue here is not a blanket rule against the coordinator writing code, but that this specific task is too large and cross-cutting for a single agent to handle effectively.

D The coordinator's API rate limits would be exceeded when processing 40+ files

API rate limits are a practical concern but not the architectural reason to delegate. The root issue is attention dilution — quality degrades when one agent processes too many items in a single context, regardless of rate limits.

C The coordinator cannot access file system tools, so it physically cannot make code changes

There is no inherent restriction preventing a coordinator from having file system tools. The issue is architectural — a single agent processing too many files suffers from attention dilution, not tool access limitations.

B Loading 40+ files into one context dilutes attention — the coordinator should delegate to scoped subagents instead

Large, multi-file tasks suffer from attention dilution when processed in a single context. Delegating to specialist subagents ensures each agent works with focused context. The coordinator should handle task decomposition and context injection, delegating implementation to specialists.

WHERE THIS COMES FROM

Lesson 1.6: Task Decomposition Strategies (Attention dilution)

Lesson 1.2: Multi-Agent Orchestration (Coordinator responsibilities)

Q22 · D5 5.1 CONTEXT-WINDOW-MANAGEMENT / FACTS-BLOCK q-5-1-006 · Enterprise Data Platform with Federated Queries

CORRECT

An agent joins Snowflake revenue data with PostgreSQL customer data via federated queries. To manage context, it progressively summarises earlier results between rounds. After three rounds it reports 'revenue increased in Q3' but cannot give the exact figure ($14.2M), which was lost in summarisation. The user asks for the precise number. What is the most effective fix?

C Increase the context window size so that summarisation is triggered less frequently.

A larger context window delays the problem but does not solve it. As queries accumulate, summarisation will eventually compress the figures. The structural fix is to extract key data from the summarisation pipeline entirely.

D Instruct the agent to always include exact figures in its summaries rather than qualitative statements.

Prompt-based instructions for summarisation behaviour are unreliable. The model compresses content probabilistically, and instructions cannot guarantee that specific numbers survive multiple rounds of summarisation.

B Extract key numerical facts into a persistent facts block outside the summarised history.

The progressive summarisation trap destroys numerical precision when conversation history is compressed. A persistent facts block preserves exact figures, dates, and identifiers across summarisation rounds, directly solving the data loss problem.

A Re-run the original Snowflake query to retrieve the exact Q3 revenue figure from the source.

Re-querying works as a recovery tactic but does not prevent the problem from recurring. The agent will summarise again on the next round, losing the figure again. A structural fix is needed.

WHERE THIS COMES FROM

Lesson 5.1: Context Window Management (Progressive summarisation)

Lesson 5.1: Context Window Management (Persistent facts)

Q23 · D2 2.2 STRUCTURED-ERROR-RESPONSES / ACCESS-VS-EMPTY q-2-2-008 ·
Enterprise Data Platform with Federated Queries CORRECT

The data platform's fetch_api tool calls a third-party pricing API. When the API key has expired, the tool returns { "data": [], "status": "ok" } instead of signalling an authentication failure. The agent tells the user 'No pricing data is available for that product' and moves on. What is the root cause?

A The agent should be instructed via the system prompt to treat empty arrays as errors and retry with exponential backoff.

Not all empty arrays are errors — a product with no pricing data is a valid empty result. The problem is that the tool masks an authentication failure as a successful empty response. System prompt instructions cannot reliably distinguish the two.

D The agent's context window is too small to hold the full API response, so it receives a truncated version that appears empty.

Context window truncation would not produce a well-formed { "data": [], "status": "ok" } response. The problem is semantic — the tool is returning a misleading success status for an authentication failure.

C The third-party API is poorly designed. The platform team should switch to a different pricing provider that returns proper HTTP error codes.

Whilst the upstream API behaviour is unhelpful, the platform team controls the MCP tool layer. The tool should detect the authentication failure (even from the upstream response) and translate it into a structured MCP error rather than passing through the misleading response.

B The tool cannot distinguish an access failure (expired key) from a valid empty result, so the agent treats a permission error as no data.

The tool returns the same structure for two fundamentally different outcomes: 'I could not access the data' versus 'I accessed the data and found nothing.' The agent cannot distinguish these and accepts the empty result at face value. The tool must return a structured error with the MCP isError flag for authentication failures.

WHERE THIS COMES FROM

Lesson 2.2: Structured Error Responses (Access failure vs empty result)

Q24 · D2 2.5 BUILT-IN-TOOLS / GREP-VS-GLOB q-2-5-008 · Enterprise Data Platform with Federated Queries CORRECT

A developer is investigating a data inconsistency in the platform codebase. They need to find all files that reference a specific Snowflake table name 'fact_revenue_daily' and then check the directory structure for related migration files. Which tool sequence is correct?

A Glob for '**/*fact_revenue_daily*' to find references, then Glob for '**/migrations/**' to find migration files.

Glob matches file paths by name, not file contents. It would find files named after the table (unlikely) but miss all files that reference the table name in their code.

C Grep for 'fact_revenue_daily' to find all content references, then Glob for '**/migrations/*.sql' to find migration files by path pattern.

Grep searches file contents — correct for finding code that references the table name. Glob matches file paths — correct for finding migration files by directory pattern. This is the optimal two-step sequence using each tool for its strength.

D Grep for 'fact_revenue_daily' for both steps — first to find references, then to find migration files containing the table.

Using Grep for both steps would find migration files that mention the table, but would miss migration files related to the table that do not contain its name (e.g. schema migrations, index changes). Glob is the correct tool for path-based file discovery.

B Read all SQL files in the project to search for the table name, then Read all migration files.

Reading all files upfront is a context-budget anti-pattern. It wastes tokens on irrelevant files and is exactly the approach the exam penalises.

WHERE THIS COMES FROM

Lesson 2.5: Built-in Tools (Grep vs Glob)

Q25 · D3 3.1 CLAUDE-MD-HIERARCHY / POSTTOOLUSE-FORMATTING q-3-1-016 ·

Large-Scale Codebase Refactoring with Multi-Agent Claude Code CORRECT

The team wants to ensure that every Java file written by Claude Code during refactoring is automatically formatted with the project's Checkstyle rules before being saved. Developers occasionally forget to run the formatter manually. What is the correct hook configuration?

B A PostToolUse hook on file write operations that runs the Checkstyle formatter on the written file, automatically correcting any style violations

PostToolUse fires after the file is written to disk. A hook that runs the Checkstyle formatter on the output file ensures every written Java file conforms to the project's style rules, regardless of what the model generated. This provides deterministic enforcement without relying on prompt instructions.

A A PreToolUse hook on the Write tool that runs Checkstyle on the content before the file is written

PreToolUse fires before the tool executes. The file has not been written yet, so there is no file on disk to format. PreToolUse is suited for blocking or validating inputs, not for post-processing outputs.

C Add 'Always run Checkstyle before saving files' to the CLAUDE.md system prompt

Prompt instructions are probabilistic. The question states developers 'occasionally forget', and an LLM-based instruction has the same failure mode. A hook provides deterministic enforcement that cannot be skipped.

D A PreToolUse hook on the Read tool that verifies all Java files are Checkstyle-compliant before they are read into context

Checking files on read does not address the requirement. The goal is to format files when they are written, not to validate files when they are read. Existing legacy files may not be Checkstyle-compliant and should not block reading.

WHERE THIS COMES FROM

Claude Code: Hooks ↗

Anthropic: Agent SDK Hooks ↗

Q26 · D4 4.2 FEW-SHOT-PROMPTING / CONSTRUCTION q-4-2-006 · CI/CD Pipeline Integration CORRECT

Your Claude Code agent generates API endpoint implementations. You provide detailed instructions specifying error handling conventions, but the generated code inconsistently handles async errors: sometimes using try/catch, sometimes using .catch(), and sometimes omitting error handling entirely. Adding more detailed instructions did not resolve the inconsistency. What is the most effective next step?

B Add 2-4 few-shot examples demonstrating the correct error handling pattern across varied async scenarios, with reasoning for each choice.

When detailed instructions alone fail to produce consistent formatting, few-shot examples with reasoning are the correct escalation. Covering varied async scenarios prevents the model from pattern-matching only the specific cases shown and teaches it to generalise the error handling style.

A Add a linting rule that flags and rejects generated code whose error handling deviates from the target style, catching nonconforming output after generation.

Rejecting nonconforming output gates results after generation but never teaches the model to produce the consistent style itself, so novel cases keep failing. Few-shot examples with reasoning fix the inconsistency at source.

D Add a post-generation review step that rewrites any incorrect error handling patterns

Post-processing adds cost and complexity. It is more effective to teach the model the correct pattern upfront with few-shot examples than to fix incorrect output afterwards.

C Set temperature to 0 to eliminate the randomness causing inconsistent error handling styles

Temperature does not fix ambiguous criteria. If the instructions do not clearly demonstrate the preferred pattern, low temperature just makes the model consistently pick the same ambiguous interpretation, not necessarily the correct one.

WHERE THIS COMES FROM

Lesson 4.2: Few-Shot Prompting (Effective examples)

Anthropic: Multishot (Few-Shot) Prompting ↗

Q27 · D3 3.6 CICD-INTEGRATION / WORKTREE-PARALLEL q-3-6-010 · Technical Documentation Maintenance System

CORRECT

A documentation team needs to simultaneously update API reference docs for three independent microservices after a breaking change. Each update requires reading source code, updating Markdown files, and validating links. A single Claude Code session would exhaust the context window trying to hold all three services' code simultaneously. What is the recommended approach?

D Split the documentation files into smaller chunks and process each chunk in a separate API call using the batch API

The batch API is for high-throughput, latency-tolerant workloads — not for interactive Claude Code sessions. Documentation updates require interactive exploration of source code and iterative editing, which the batch API does not support.

B Use git worktree for three branches, each with its own Claude Code session on one service, then merge the results

git worktree creates separate working directories on different branches, each with its own isolated Claude Code session. Each session has a full context budget dedicated to one service. The three updates run in parallel without interfering with each other, and results are merged via standard git workflows.

A Process the three services sequentially in the same session, running /compact between each service to free context

Even with /compact, each service's context (source code, documentation, edits) is substantial. Sequential processing risks context degradation as compaction summaries lose detail from earlier services, and it is slower than parallel processing.

C Create a single skill with context: fork that processes all three services in parallel within one session

context: fork isolates a skill's output from the main conversation but does not create multiple parallel execution contexts. A single skill cannot simultaneously process three independent services in parallel.

WHERE THIS COMES FROM

Anthropic: Claude Code Documentation ↗

Git: Worktrees ↗

Q28 · D2 2.3 TOOL-DISTRIBUTION-CHOICE / CONSOLIDATION q-2-3-008 · Enterprise Data Platform with Federated Queries

CORRECT

The data platform team has built an MCP server with 22 tools: query_snowflake, query_postgres, query_api, plus 19 specialised tools for individual data transformations (pivot_table, calculate_percentile,

normalise_currency, etc.). Agents take 3-4 turns to select the correct tool and frequently choose the wrong transformation. What is the most effective redesign?

D Split the tools across two separate MCP servers — one for queries and one for transformations — to reduce cognitive load.

MCP server boundaries are invisible to the agent. All tools from all connected servers appear in a single list. Splitting across servers does not reduce the number of tools the agent must choose from.

B Consolidate the 19 transformation tools into a single transform_data tool with a transform_type parameter, reducing the total to 4 tools.

Reducing from 22 to 4 tools brings the count within the optimal 4-5 range for reliable tool selection. The 19 transformation tools share a common pattern (input data, transformation type, output format) and are natural candidates for consolidation into a single parameterised tool.

C Use tool_choice: 'any' to force the agent to always call a tool, eliminating turns where the agent reasons without acting.

Forcing tool calls does not improve selection accuracy — it just ensures the agent picks something, potentially the wrong tool. The root cause is too many similar tools, not too few tool calls.

A Improve all 22 tool descriptions with detailed examples and boundary conditions to help the agent distinguish between them.

Better descriptions help, but 22 tools still exceeds the practical limit for reliable selection. Research shows tool selection degrades significantly beyond 4-5 tools per agent. The tool count itself is the problem.

WHERE THIS COMES FROM

Lesson 2.3: Tool Distribution and Tool Choice (Consolidating near-duplicate tools)

Lesson 2.3: Tool Distribution and Tool Choice (Tool overload)

Q29 · D4 4.5 BATCH-PROCESSING / BATCH-FAILURE q-4-5-002 · CI/CD Pipeline Integration CORRECT

Your CI/CD pipeline generates nightly security audit reports using the Message Batches API. A batch of 200 documents completes overnight, but 15 documents fail due to exceeding the context window. The team proposes resubmitting the entire batch of 200 documents. What is the correct failure handling strategy?

C Switch the entire pipeline to synchronous processing to avoid batch failures

Nightly security audits are latency-tolerant workloads perfectly suited for batch processing. Switching to synchronous forfeits the 50% cost savings without solving the oversized document problem.

D Increase the batch processing timeout to give the system more time to handle large documents

The failure is due to documents exceeding the context window, not a timeout issue. The documents need to be chunked to fit within the model's context limits.

B Resubmit only the 15 failed documents identified by custom_id, chunking the oversized documents before resubmission

Identify failed documents by custom_id, then resubmit only failures with modifications (chunking oversized documents). This is the correct batch failure handling pattern — it avoids reprocessing the 185 successful documents and addresses the root cause of the failures.

A Resubmit the entire batch since you cannot identify which documents failed

The batch API uses custom_id fields to correlate request/response pairs. Failed documents can be identified by their custom_id, making full resubmission unnecessary and wasteful.

WHERE THIS COMES FROM

Lesson 4.5: Batch Processing and Prompt Optimisation (Batch failure handling)

Lesson 4.5: Batch Processing and Prompt Optimisation (custom_id matching)

Q30 · D1 1.5 AGENT-SDK-HOOKS / POSTTOOLUSE-HOOKS q-1-5-012 CORRECT

A customer support agent must redact credit card numbers from its responses before they reach the customer. The current system prompt instructs the agent to replace card numbers with asterisks, but QA testing reveals that 6% of responses still contain unredacted card numbers. What guardrail approach should the architect use?

C Implement a PreToolUse hook that blocks any tool call containing a credit card number in its parameters

PreToolUse hooks intercept outgoing tool calls, not incoming results. The credit card numbers appear in tool results (data returned from backend systems), not in tool call parameters. PostToolUse is the correct hook type for redacting data from results.

D Remove all tools that might return credit card numbers from the agent's allowedTools

Removing tools that return card numbers would eliminate the agent's ability to look up order and payment information, which is core to its customer support function. The correct approach is to keep the tools but redact sensitive data from their results.

B Add a PostToolUse hook that regex-redacts credit card numbers from tool results before the model sees them.

A PostToolUse hook runs deterministic code (regex pattern matching) on tool results before the model sees them. This ensures credit card numbers are redacted with 100% reliability, regardless of model behaviour. This is the correct guardrail for data security requirements.

A Rewrite the system prompt with more explicit redaction instructions and add few-shot examples of correct redaction

The current prompt already instructs redaction but fails 6% of the time. Strengthening prompts may reduce the failure rate but cannot eliminate it. Leaking credit card numbers is a data security issue that demands deterministic enforcement.

WHERE THIS COMES FROM

Lesson 1.5: Agent SDK Hooks (PostToolUse hooks)

Anthropic: Agent SDK Hooks ↗

Q31 · D4 4.1 SYSTEM-PROMPTS / KEYWORD-OVERLAP q-4-1-004 · CI/CD Pipeline Integration CORRECT

Your CI/CD pipeline uses a system prompt with a tool named 'check_security' and also includes instructions that say 'check the security of each function'. Developers report that the model sometimes produces a text-based security analysis instead of calling the check_security tool. What is the most likely cause?

A The model's temperature is too high, causing random tool selection

Temperature affects randomness in generation but is not the primary cause of tool call confusion. The issue is prompt design, not sampling parameters.

C The model cannot call tools from within a CI/CD pipeline context

Tool calling works in any context. The issue is prompt design, not an environmental limitation.

D The system prompt has grown too long, so the model skips over the tool definitions near the end and never registers that check_security is available to call.

System prompt length does not cause tool definitions to be skipped. The model processes tools and instructions together. The issue is keyword overlap creating ambiguity.

B The 'check the security' instruction keyword-overlaps the 'check_security' tool name, so the model follows the text instead of calling the tool.

When system prompt instructions use phrasing that closely mirrors tool names, the model can interpret 'check the security' as a general instruction rather than a trigger for the check_security tool. Distinct naming reduces this ambiguity.

WHERE THIS COMES FROM

Lesson 4.1: System Prompts with Explicit Criteria (Keyword overlap)

Anthropic: Tool Use Documentation ↗

Q32 · D3 3.1 CLAUDE-MD-HIERARCHY / POSTTOOLUSE-ENFORCEMENT q-3-1-014 · CI/CD Pipeline Integration INCORRECT

A fintech company requires that all API response payloads are normalised to snake_case before being logged, and that any file write to the src/api/ directory is automatically linted. They want these enforcements to be deterministic and not rely on the model remembering instructions. Which hook configuration achieves both requirements?

B A PostToolUse hook on file writes to src/api/ that runs the linter on the created file, and a PostToolUse hook on data processing that normalises response payloads to snake_case

Both are PostToolUse hooks because both act on completed outputs. The linter runs after a file write to src/api/ is complete, validating the written content. The normalisation hook processes API response data after it is retrieved, converting fields to snake_case before logging. Both provide deterministic enforcement independent of model instructions. (correct answer)

C Add both requirements to CLAUDE.md with clear examples, and add a PreToolUse hook that reminds the model about these rules before each tool call

CLAUDE.md instructions and PreToolUse reminders both rely on the model's compliance, which is not deterministic. The requirement explicitly states that enforcement must not depend on the model remembering instructions.

D A PreToolUse hook that blocks any file write to src/api/ unless the file has already been linted, and a PreToolUse hook that blocks API calls unless snake_case normalisation is configured

PreToolUse blocking assumes the file exists before the write, which is contradictory. You cannot lint a file that has not been written yet. PostToolUse hooks that process output after completion are the correct approach for validation and normalisation.

A A PreToolUse hook on file writes that runs the linter before the write completes, and a PostToolUse hook on API response handling that normalises field names to snake_case

PreToolUse fires before the tool runs, so there is no file content to lint yet. Linting must happen after the file is written (PostToolUse). The ordering of hooks to actions is incorrect here. (your answer)

WHERE THIS COMES FROM

Claude Code: Hooks ↗

Q33 · D4 4.5 BATCH-PROCESSING / CUSTOM-ID q-4-5-004 · CI/CD Pipeline Integration CORRECT

Your CI/CD pipeline processes code reviews using the Message Batches API. After a batch completes, you need to match each review result back to the corresponding pull request. What is the correct mechanism for correlating batch results with their original requests?

B Rely on the batch API returning results in the same order they were submitted

Batch results are not guaranteed to return in submission order. The custom_id field is the correct correlation mechanism.

A Parse the review content to identify which pull request it refers to based on file names mentioned in the review

Content parsing is fragile and error-prone. The Batches API provides a built-in mechanism for result correlation.

D Submit each pull request as a separate batch of one item to maintain correlation through the batch ID

Submitting individual batches defeats the purpose of batch processing and does not leverage the custom_id mechanism designed for exactly this use case.

C Use the custom_id field in each batch request to store the pull request identifier, then match results by custom_id

The custom_id field exists specifically for correlating batch request/response pairs. Setting it to the pull request identifier provides reliable, direct correlation without relying on result ordering or content parsing.

WHERE THIS COMES FROM

Lesson 4.5: Batch Processing and Prompt Optimisation (custom_id matching)

Anthropic: Message Batches API ↗

Q34 · D5 5.6 INFORMATION-PROVENANCE / STRUCTURED-PROVENANCE q-5-6-008 · Technical Documentation Maintenance System CORRECT

A docs team has Claude Code generate architecture guides by synthesising source code, inline comments, commit messages, and ADRs. After several edits, the guides no longer indicate which statements came from which source. A developer later questions whether a specific architectural constraint in a guide is still valid. What approach preserves source attribution?

A Instruct Claude Code to add footnotes to the generated documentation citing the original source for each statement

Prose footnotes are fragile — they are lost or degraded during subsequent editing passes, summarisation, or reformatting. The team has already experienced this problem through 'several iterations of editing.'

B Maintain structured provenance metadata that maps each claim to its source file, line number, and retrieval date.

Structured provenance metadata survives editing because it is stored as data fields (e.g., JSON or YAML frontmatter) separate from the prose. When a developer questions a claim, they can trace it to the exact source file and line number. The retrieval date indicates whether the source was current when the documentation was generated.

D Version-control the documentation and use git blame to trace each line back to the commit that created it

git blame traces authorship of documentation lines to commits, not to the source material that informed those lines. A commit message might say 'update architecture guide' without indicating that a specific statement came from ADR-047 or a comment in auth-service/src/middleware.ts.

C Keep all original source files unchanged in a reference directory so developers can manually trace any claim back to its origin

Preserving source files without explicit mappings forces developers to manually search through potentially hundreds of files to verify a single claim. This is not scalable and does not indicate which specific source informed which specific documentation statement.

WHERE THIS COMES FROM

Lesson 5.6: Information Provenance and Multi-Source Synthesis (Attribution preservation)

Lesson 5.6: Information Provenance and Multi-Source Synthesis (Structured claim-source mappings)

Q35 · D2 2.3 TOOL-DISTRIBUTION-CHOICE / ROLE-SCOPING q-2-3-007 · Enterprise Data Platform with Federated Queries CORRECT

An enterprise data platform agent has 22 tools available, including query tools for Snowflake, PostgreSQL, and third-party APIs, plus data transformation, visualisation, export, and administrative tools. The team notices the agent frequently selects the wrong query tool — choosing the Snowflake tool for data that lives in PostgreSQL. What is the most effective architectural fix?

B Split the agent into role-specific agents — a query agent with 4-5 data source tools, a transformation agent, and an export agent — each scoped to its specialisation.

Tool overload at 22 tools degrades selection reliability. Splitting into role-specific agents with 4-5 tools each reduces decision complexity and scopes each agent to its specialisation. The query agent can focus on selecting the right data source without being distracted by transformation or export tools.

D Consolidate all query tools into a single universal_query tool that internally routes to the correct data source based on the query syntax.

A universal query tool hides the data source distinction from the agent. When the agent needs to reason about which data source contains the relevant data, it loses the ability to make that decision explicitly.

C Add a pre-processing step that analyses the user query to determine which data source is needed, then use tool_choice forced selection to call the correct query tool.

Forced tool selection based on a pre-processing step is over-engineered and brittle. It bypasses the LLM's ability to reason about which tool to use and breaks when queries span multiple data sources.

A Improve the descriptions of all 22 tools to include explicit boundaries and data source mappings.

Task Statement 2.1 does teach descriptions as the first fix for misrouting, which is why this option is tempting, but that rule assumes the agent has a workable number of tools and simply cannot tell two of them apart. Here there are 22, well past the 4-5 range where selection stays reliable. Decision complexity degrades selection regardless of how well each description is written, so rewriting all 22 treats the symptom and leaves the cause.

WHERE THIS COMES FROM

Lesson 2.3: Tool Distribution and Tool Choice (Role-specific scoping)

Lesson 2.3: Tool Distribution and Tool Choice (Tool overload)

Q36 · D1 1.7 SESSION-STATE-RESUMPTION / FRESH-START q-1-7-004 · Large-Scale Codebase Refactoring with Multi-Agent Claude Code CORRECT

A Claude Code agent has been working on a feature branch for 45 minutes and has accumulated extensive context about the codebase. The developer notices that several tool results from early in the session (file reads from 40 minutes ago) are now stale because a colleague pushed changes to those files. The agent is making recommendations based on the outdated file contents. What is the best recovery strategy?

A Continue in the current session and simply ask the agent to re-read the files a colleague changed, so it picks up the latest contents.

Re-reading updates the file contents, but the stale results remain in the context window. The agent may still reference the outdated content from earlier in the conversation, leading to contradictory reasoning.

C Start a fresh session with a summary of the key findings and decisions, then read the changed files for current state.

Fresh start with summary injection is the prescribed recovery strategy for stale tool results. It preserves the valuable insights and decisions from the previous session while eliminating the stale data. Reading the changed files in the new session ensures the agent works from current state.

D Use fork_session to create a new branch that excludes the stale tool results

fork_session creates a branch from the current state, which includes the stale tool results. It does not allow selective exclusion of context. It is designed for divergent exploration, not for stale context recovery.

B Start a completely new session with no context from the previous session

Starting fresh discards 45 minutes of accumulated context and architectural understanding. The agent would need to rediscover all the insights it already found about the codebase.

WHERE THIS COMES FROM

Lesson 1.7: Session State and Resumption (Stale context)

Lesson 1.7: Session State and Resumption (Decision matrix)

Q37 · D1 1.2 ORCHESTRATION-PATTERNS / CONTEXT-PASSING q-1-2-007 ·

Large-Scale Codebase Refactoring with Multi-Agent Claude Code CORRECT

A team runs a coordinator agent that delegates refactoring work to three specialist subagents: a schema-migration agent, an API-layer agent, and a test-update agent. The test-update agent frequently produces tests that reference database schemas the schema-migration agent has already renamed. The subagents do not communicate with each other. What is the root cause?

A The subagents need a shared message bus so the test-update agent can query the schema-migration agent for the latest schema names

In a hub-and-spoke multi-agent architecture, subagents must never communicate directly. All communication flows through the coordinator. Adding a message bus violates this principle and creates coordination complexity.

D The schema-migration agent and test-update agent should run sequentially rather than in parallel to avoid race conditions

Sequencing alone does not solve the problem if the coordinator does not pass the schema-migration agent's results to the test-update agent. The issue is missing context injection, not execution order.

C The coordinator is not passing the schema-migration agent's output (including renamed schemas) as context when delegating to the test-update agent

In hub-and-spoke orchestration, the coordinator manages all inter-agent communication. Subagents do not inherit each other's context. The coordinator must explicitly pass relevant outputs from earlier subagents as context to downstream subagents. Here, the renamed schema mappings must be injected into the test-update agent's context.

B The test-update agent should have read access to the schema migration files so it can discover the new names itself

While this might work as a workaround, it misses the architectural root cause. The coordinator is responsible for passing all necessary context to each subagent. The test-update agent should not need to discover information that the coordinator already has.

WHERE THIS COMES FROM

Lesson 1.2: Multi-Agent Orchestration (Isolation principle)

Lesson 1.3: Subagent Invocation and Context Passing (Context passing)

Q38 · D1 1.5 AGENT-SDK-HOOKS / HOOK-PLACEMENT q-1-5-008 ·

Large-Scale Codebase Refactoring with Multi-Agent Claude Code CORRECT

The team's refactoring pipeline uses a PreToolUse hook to enforce that every new microservice includes a Dockerfile. However, the hook only checks for Dockerfile existence at the moment the create_service tool is called. A developer points out that some services have their Dockerfiles added in a later commit by a different subagent. What is the flaw in this approach?

C Run the hook on every single file write and continuously re-check whether a Dockerfile has appeared yet, failing the build until one does.

Running a Dockerfile check on every file write is wasteful and creates unnecessary overhead. The check should happen once at the right boundary (deployment or merge), not continuously during development.

A The hook should use PostToolUse instead of PreToolUse, checking after service creation whether a Dockerfile was added

PostToolUse checks after the tool executes but still only at that single point in time. If the Dockerfile is added in a later commit by a different subagent, PostToolUse on create_service will still miss it.

B Gate the deployment or merge tool rather than create_service, checking Dockerfile existence when the service is considered complete.

Compliance checks should be enforced at the boundary where the precondition must hold — in this case, before deployment or merge. Checking at service creation time is too early when the workflow allows Dockerfiles to be added in subsequent steps. Moving the gate to the deployment or merge boundary ensures all required artefacts are present regardless of which subagent adds them.

D The PreToolUse hook is correct but needs a timeout to wait for the Dockerfile to appear within a configurable window

Adding a timeout introduces non-deterministic behaviour and race conditions. The correct fix is to move the enforcement gate to the right point in the workflow, not to add polling delays.

WHERE THIS COMES FROM

Lesson 1.5: Agent SDK Hooks (Decision framework: where to enforce)

Anthropic: Agent SDK Hooks ↗

Q39 · D2 2.4 MCP-SERVER-INTEGRATION / BUILD-VS-USE q-2-4-007 · Enterprise Data Platform with Federated Queries CORRECT

An enterprise data platform team is evaluating how to integrate with their existing PostgreSQL database, Slack workspace, and a custom internal approval workflow. A developer proposes building three custom MCP servers. What is the correct approach?

D Use community MCP servers for all three integrations, adapting the internal approval workflow to fit an existing community server's interface.

Forcing a custom internal workflow into a community server's interface leads to awkward abstractions and missing functionality. The community-first principle applies to standard integrations, not to genuinely unique team-specific workflows.

C Skip MCP entirely and use direct API calls from Bash for all three integrations to avoid the overhead of running MCP servers.

Direct API calls bypass the MCP tool interface, losing the benefits of structured tool descriptions, standardised error handling, and agent-native integration. MCP servers provide a consistent interface that agents understand natively.

B Use community MCP servers for PostgreSQL and Slack, and build a custom server only for the internal approval workflow that has no community equivalent.

Community servers should be evaluated first for standard integrations. PostgreSQL and Slack have well-maintained community MCP servers. The internal approval workflow is team-specific with no community equivalent, making it the only justified custom build.

A Build all three custom MCP servers to ensure tight integration with the team's specific requirements and coding standards.

Building custom servers for PostgreSQL and Slack is unnecessary when well-maintained community servers already exist for these standard integrations. Custom builds should be reserved for genuinely unique workflows.

WHERE THIS COMES FROM

Lesson 2.4: MCP Server Integration (Build-vs-use)

Claude Code: MCP Server Configuration ↗

Q40 · D5 5.1 CONTEXT-WINDOW-MANAGEMENT / TOOL-RESULT-TRIMMING q-5-1-005 ·
Enterprise Data Platform with Federated Queries CORRECT

An analytics agent queries Snowflake and receives 40+ columns per row, only 5 of which are relevant to the user's question about quarterly revenue. The agent appends the full result set to context. After three such queries the window is nearly full and follow-up questions fail. What is the most effective fix?

B Trim tool results to only the relevant columns before appending them to the conversation context.

Tool result trimming removes irrelevant fields before they enter the context window. Keeping only the 5 relevant columns from each 40+ column result set dramatically reduces token consumption, allowing the agent to handle many more queries within the same context budget.

C Store all query results in an external database and have the agent retrieve specific values on demand instead of keeping results in context.

External storage adds infrastructure complexity without addressing the core issue. The agent still needs some result data in context to reason about it. Trimming irrelevant fields is simpler and directly reduces context consumption.

A Upgrade to a model with a larger context window so that full result sets can be accommodated across more queries.

A larger context window postpones the problem but does not solve it. Each full result set still wastes tokens on 35+ irrelevant columns, and the context will eventually fill regardless of size.

D Limit the number of rows returned by each query to reduce the total data volume in context.

Row limits reduce data volume but may exclude relevant rows. The problem is column-level verbosity (35+ irrelevant columns per row), not row count. Trimming columns preserves all relevant rows whilst eliminating irrelevant fields.

WHERE THIS COMES FROM

Lesson 5.1: Context Window Management (Tool result trimming)

Lesson 5.1: Context Window Management (Upstream optimisation)

Q41 · D3 3.6 CICD-INTEGRATION / STRUCTURED-VALIDATION q-3-6-007 · Technical Documentation Maintenance System
CORRECT

A CI pipeline validates that generated API docs match the codebase. It runs `claude -p 'Verify that all public API endpoints in src/api/ have corresponding documentation in docs/api/'` with `--output-format json`. The check consistently reports 100% coverage, yet manual audits regularly find undocumented endpoints. What is the most likely cause?

D The CI pipeline needs a separate review step where a second Claude Code instance verifies the first instance's findings

Adding a second instance to review the first adds latency and cost without addressing the root cause. If the first instance's prompt lacks clear validation criteria, the second instance reviewing the same vague output will draw the same flawed conclusions.

A The --output-format json flag corrupts the analysis results, causing false positives

--output-format json controls the output structure, not the analysis logic. It formats results as JSON for downstream parsing but does not affect the accuracy of the verification itself.

B Claude Code is doing a shallow pattern match, not a deep semantic comparison, and the prompt lacks validation criteria

Without explicit validation criteria (e.g., 'for each exported function in src/api/, check that docs/api/ contains a section with the function name, parameters, and return type'), the model may perform a shallow check. Adding structured criteria, expected output format, and examples of what constitutes a gap produces reliable verification.

C The -p flag prevents Claude Code from reading files, so it generates a plausible-sounding report without actual file access

The -p flag runs Claude Code in non-interactive (pipe) mode. It does not restrict file access — Claude Code retains full access to its built-in tools (Read, Grep, Glob) for reading files.

WHERE THIS COMES FROM

Lesson 3.6: CI/CD Integration (Structured output and validation)

Lesson 3.6: CI/CD Integration (CLAUDE.md for CI context)

Q42 · D1 1.7 SESSION-STATE-RESUMPTION / FORK-SESSION q-1-7-007 ·
Large-Scale Codebase Refactoring with Multi-Agent Claude Code CORRECT

A developer is exploring two different strategies for decomposing the monolith's order processing module: (1) splitting by business capability (orders, payments, shipping) or (2) splitting by data ownership (order-db, payment-db, shipping-db). They want to explore both approaches in parallel without losing either analysis. What is the correct Claude Code approach?

D Instruct the agent to evaluate both strategies sequentially in the same session, then compare results

Sequential evaluation in a single session means the first strategy's analysis occupies context when exploring the second, potentially biasing the comparison. fork_session allows true parallel exploration with isolated contexts.

A Open two terminal tabs and run separate Claude Code sessions with different instructions in each

Separate sessions do not share the baseline context of the current analysis. Both explorations would need to re-do the initial codebase discovery, wasting time and losing the shared foundation.

B Use fork_session to create two parallel exploration branches from the current session's baseline, one for each decomposition strategy

fork_session creates independent branches from a shared baseline. Both branches inherit the current codebase analysis and context but can diverge independently to explore different strategies. Neither branch's exploration affects the other, and both analyses are preserved.

C Use --resume to alternate between the two strategies in a single session, relying on conversation history to separate the analyses

--resume continues a single linear session. Alternating between strategies in one context risks cross-contamination where analysis from one approach influences the other. fork_session provides clean isolation for parallel exploration.

WHERE THIS COMES FROM

Lesson 1.3: Subagent Invocation and Context Passing (fork_session)

Lesson 1.7: Session State and Resumption (Session management options)

Q43 · D4 4.1 SYSTEM-PROMPTS / SEVERITY-EXAMPLES q-4-1-002 · CI/CD Pipeline Integration CORRECT

Your Claude Code review prompt for the pipeline's polyglot codebase classifies severity using prose descriptions like 'critical means the code is dangerous' and 'minor means the code is slightly suboptimal'. Developers complain that identical code patterns receive different severity ratings across runs. What is the most effective improvement?

D Add a second model pass that re-evaluates each finding's severity to catch inconsistencies

A second pass using the same vague prose criteria will produce the same inconsistency. The criteria themselves must be improved with concrete code examples before adding verification layers.

C Lower the model temperature to 0 to ensure deterministic severity ratings

While lower temperature reduces randomness, it does not address the fundamental problem: the model has no concrete reference for what each severity level looks like. Code examples provide that reference.

A Replace prose severity descriptions with concrete TypeScript code examples for each severity level.

Severity calibration with concrete code examples produces consistent classification across invocations. Prose descriptions are inherently ambiguous and leave the model to interpret severity differently each time.

B Add a confidence threshold so only findings above 90% confidence are reported, filtering out uncertain severity ratings

Self-reported confidence scores are poorly calibrated. Confidence-based filtering does not fix the root cause, which is ambiguous severity criteria. Explicit code examples are the correct fix.

WHERE THIS COMES FROM

Lesson 4.1: System Prompts with Explicit Criteria (Severity calibration)

Q44 · D1 1.1 AGENTIC-LOOPS / ANTI-PATTERNS q-1-1-006 ·
Large-Scale Codebase Refactoring with Multi-Agent Claude Code CORRECT

A refactoring agent decomposes a legacy Java monolith into microservices. Its loop terminates by parsing the assistant's last message for the phrase 'refactoring complete', running 20 iterations even on simple classes. Logs show stop_reason returns 'end_turn' after iteration 4. What is the correct fix?

C Lower the iteration cap to 5 so the agent stops sooner for simple classes

Lowering the cap would truncate complex refactoring tasks that genuinely need more iterations. The root cause is using natural language parsing instead of the stop_reason field for loop control.

A Improve the phrase detection to also check for 'finished', 'done', and 'all changes applied'

Adding more phrases to parse is still natural language detection, which is an anti-pattern. The model may finish without using any of these phrases, or use them mid-task. The stop_reason field already provides a deterministic signal.

B Replace the natural language parsing with stop_reason inspection: exit on 'end_turn', continue on 'tool_use'.

Parsing natural language for termination is an anti-pattern. The stop_reason field is the authoritative, deterministic signal for agentic loop control. 'tool_use' means Claude has more work to do; 'end_turn' means it considers the task complete.

D Add a tool that the agent must call explicitly to signal completion, and check for that tool call each iteration

A custom completion tool adds unnecessary complexity. The API already provides the stop_reason field as a built-in, deterministic mechanism for exactly this purpose.

WHERE THIS COMES FROM

Lesson 1.1: Agentic Loops (Anti-patterns)

Q45 · D3 3.6 CICD-INTEGRATION / SESSION-ISOLATION q-3-6-005 · CI/CD Pipeline Integration CORRECT

A CI/CD pipeline runs three Claude Code steps in sequence: (1) generate a changelog from git commits, (2) review the changelog for accuracy, and (3) check for breaking changes. The team notices that step 2 never flags inaccuracies and step 3 misses obvious breaking changes that the changelog omits. What is the root cause?

A The three steps share session context, so steps 2 and 3 inherit step 1's reasoning instead of judging the changelog

When review and analysis steps share context with the generation step, they inherit the original reasoning and are less likely to identify gaps or errors. Independent sessions force each step to evaluate the changelog from scratch without bias from the generation reasoning. Session context isolation is critical for CI/CD review pipelines.

D The steps need to use --output-format json so that each step can parse the previous step's structured output

Output format affects how results are structured, not whether reviews are thorough. JSON output would help with parsing but would not address the fundamental issue of review steps retaining generation context bias.

C The -p flag is not being used, causing each step to wait for interactive input

If -p were missing, the pipeline would hang rather than produce output that misses issues. The steps are producing output (the changelog is generated, reviews complete), but the reviews are ineffective.

B The CLAUDE.md file does not contain changelog formatting standards, so the review step has no criteria to evaluate against

Missing formatting standards would affect the quality of the generated changelog, not the review's ability to catch inaccuracies. The pattern of reviews consistently missing issues points to shared-context bias, not missing criteria.

WHERE THIS COMES FROM

Lesson 3.6: CI/CD Integration (Session context isolation)

Q46 · D5 5.3 ERROR-PROPAGATION / SILENT-SUPPRESSION q-5-3-006 · Enterprise Data Platform with Federated Queries CORRECT

The data platform agent queries three sources in sequence: Snowflake (succeeds), PostgreSQL (returns a connection timeout), and a pricing API (succeeds). The agent silently skips the PostgreSQL failure and produces a report using only Snowflake and API data, without informing the user that customer data is missing. What is the critical design failure?

A The agent should retry the PostgreSQL query automatically with exponential backoff before producing the report.

Automatic retries are appropriate for transient errors, but the critical failure is that the agent silently suppresses the error. Even after retries exhaust, the agent must surface the failure to the user rather than producing an incomplete report without disclosure.

B The agent silently suppresses the error instead of propagating structured error context that annotates which data sources failed and what coverage the report actually has.

Silent error suppression is one of the most dangerous anti-patterns in multi-source systems. The agent must propagate structured error context — identifying which sources failed, what data is missing, and what coverage the final report represents — so the user can make informed decisions about the report's completeness.

D The orchestrator should pre-check all data source connections before allowing the agent to begin querying.

Pre-checking connections does not prevent runtime failures (a source can become unavailable between the check and the query). The agent must handle failures gracefully at query time with structured error propagation.

C The agent should abort the entire report if any single data source fails, to avoid producing incomplete results.

Aborting on any failure is overly conservative. Partial results with clear coverage annotations are often more useful than no results. The fix is transparency about what is missing, not refusing to produce anything.

WHERE THIS COMES FROM

Lesson 5.3: Error Propagation in Multi-Agent Systems (Silent suppression anti-pattern)

Lesson 5.3: Error Propagation in Multi-Agent Systems (Structured error context)

Q47 · D3 3.6 CICD-INTEGRATION / CLAUDE-MD-CRITERIA q-3-6-006 ·

Large-Scale Codebase Refactoring with Multi-Agent Claude Code INCORRECT

The team's CLAUDE.md specifies that all new microservices must include integration tests that verify API contract compliance and database migration rollback. A developer notices that Claude Code sometimes generates unit tests that mock the database instead of writing the required integration tests. What should the team add to their configuration?

B Tighten the CLAUDE.md test rule with criteria like real DB connections and contract assertions

CLAUDE.md is the correct location for persistent test standards. Specifying concrete criteria (real database connections, API contract assertions) with examples of expected structure and fixtures gives Claude Code an unambiguous target. This addresses the root cause: the existing instructions were not specific enough about what 'integration tests' means in this codebase. (correct answer)

D A .claude/rules/ file with paths: ["**/*Test.java"] that instructs Claude Code to never use mocks

Banning mocks from all test files is overly broad — mocks are appropriate in many unit tests. The requirement is specifically that integration tests with real connections must be included, not that mocks must be eliminated.

A A PostToolUse hook that rejects any test file containing mock annotations

Rejecting all mocks is too aggressive — unit tests with mocks are still valuable for other purposes. The issue is that integration tests are missing, not that unit tests exist. The configuration should specify what is required, not ban useful testing patterns. (your answer)

C A separate testing subagent that reviews generated tests and rewrites any that use mocks

A review subagent adds complexity and is still probabilistic. Clear specifications in CLAUDE.md solve the issue at the source by telling Claude Code exactly what to generate, rather than correcting after generation.

WHERE THIS COMES FROM

Lesson 3.6: CI/CD Integration (CLAUDE.md for CI context)

Q48 · D5 5.5 HUMAN-REVIEW-CALIBRATION / AGGREGATE-METRICS-TRAP q-5-5-006 ·

Enterprise Data Platform with Federated Queries CORRECT

The data platform team runs a human review of the agent's federated query reports. Reviewers sample 50 reports and find an overall accuracy rate of 97%. Management approves the system for production. Three weeks later, users report that currency conversion calculations in cross-border revenue reports are wrong 40% of the time. How did the review process fail?

C The reviewers were not domain experts in currency conversion and could not identify the errors.

Reviewer expertise is a valid concern but is not the structural failure. Even expert reviewers would miss the problem if the sample did not include enough cross-border reports. The sampling methodology is the root cause.

B The reviewers used aggregate accuracy metrics that masked category-specific failures such as the 40% currency error rate.

This is the aggregate metrics trap. When a low-frequency report type (cross-border revenue) has high error rates, the overall accuracy metric is dominated by high-frequency, easy report types. Stratified random sampling across report categories would have caught the currency conversion failures.

A The sample size of 50 reports was too small to detect a 40% error rate in currency conversions.

A 40% error rate in any category should be detectable in 50 reports — but only if the sample includes enough reports from that category. The problem is sampling strategy, not sample size.

D The agent's accuracy degraded over time due to model drift, and the initial 97% rate was genuinely correct at the time of review.

LLMs served via API do not experience gradual model drift within a three-week window. The error pattern was present during the review but hidden by aggregate metrics that did not stratify by report category.

WHERE THIS COMES FROM

Lesson 5.5: Human Review and Confidence Calibration (Aggregate metrics trap)

Q49 · D5 5.4 CODEBASE-EXPLORATION / SCRATCHPAD q-5-4-009 · Technical Documentation Maintenance System CORRECT

A docs team asks Claude Code to audit documentation coverage across a 500,000-line codebase. After reading 30 files via the Read tool, the conversation is approaching the context limit and earlier tool results will be summarised away by auto-compaction. The agent still needs that earlier information to complete the audit. What is the best strategy?

C Process all files in a single Read tool call by passing a glob pattern, so the results arrive in one untrimmed block

The Read tool reads individual files, not glob patterns. Even if multiple files could be read at once, a single massive result block would itself be trimmed or would consume the entire context budget, preventing the agent from reasoning about the results.

D Run /compact after every 10 files to free up context space for the next batch of file reads

/compact summarises the conversation, which may discard the specific findings from earlier file reads. The problem is preserving information, not just freeing space. Compacting without first persisting findings to disk loses the very data the audit needs.

A Increase the context window size so that all 30+ file contents can be held simultaneously without trimming

The context window has a fixed maximum size. A 500,000-line codebase cannot fit entirely in context regardless of configuration. Tool result trimming exists specifically because holding all file contents simultaneously is not feasible.

B Have the agent write structured findings to a scratchpad after each file, then read it for the final audit.

A scratchpad file persists on disk and can be re-read at any time, unlike in-context tool results that are trimmed as new results arrive. By extracting and recording the essential findings (not the full file contents) after each file read, the agent decouples its knowledge from context window limits. The final audit reads the compact scratchpad rather than needing all source files in context.

WHERE THIS COMES FROM

Lesson 5.4: Codebase Exploration and Context Degradation (Scratchpad files)

Lesson 5.4: Codebase Exploration and Context Degradation (/compact behaviour)

Q50 · D1 1.7 SESSION-STATE-RESUMPTION / FRESH-START q-1-7-005 · Large-Scale Codebase Refactoring with Multi-Agent Claude Code CORRECT

A Claude Code agent has spent 30 minutes debugging a failing test suite mid-refactor, trying three different approaches that each modified configuration files. None worked, and its context now holds three sets of conflicting modifications and failed outputs. The developer wants to try a completely different strategy. What session management approach should they use?

D Use fork_session from the point before the first debugging attempt to explore the new strategy

Forking from 30 minutes ago creates a branch without the knowledge of what was tried and why it failed. The new strategy benefits from knowing that three other approaches failed. Summary injection in a fresh session is more appropriate.

C Start a fresh session summarising the three failed approaches and why each failed, then pursue the new strategy with clean context.

Fresh start with summary injection preserves the knowledge of what has been tried (preventing repetition) while eliminating the polluted context from three sets of conflicting file modifications. The agent starts with clean context and the lessons learned from previous attempts.

B Start a completely new session with no prior context at all, re-read the failing tests from scratch, and apply the new strategy fresh.

A completely blank session discards the useful record of what has already been tried and why it failed, which risks repeating failed approaches. Summary injection preserves that knowledge.

A Continue in the current session and ask the agent to ignore all previous attempts and start fresh

Asking the agent to ignore context is unreliable. The three sets of conflicting modifications and failed outputs remain in the context window, and the model may still reason from them. This is a stale/polluted context problem.

WHERE THIS COMES FROM

Lesson 1.7: Session State and Resumption (Stale context)

Lesson 1.7: Session State and Resumption (Decision matrix)

Q51 · D3 3.2 SLASH-COMMANDS-SKILLS / SKILLS-PLACEMENT q-3-2-014 ·
Large-Scale Codebase Refactoring with Multi-Agent Claude Code CORRECT

The team creates a reusable /extract-service skill that guides Claude Code through the standard steps of extracting a module from the monolith into a microservice (identify boundaries, create service scaffold, migrate code, update call sites, add tests). Where should this skill be stored so that every team member who clones the repository has access?

C In the repository-root CLAUDE.md as an inline procedure

CLAUDE.md is for always-loaded standards, not on-demand workflows. An extraction procedure is invoked when needed, not applied to every session. Placing it in CLAUDE.md wastes tokens when developers are not extracting services.

A In ~/.claude/skills/ on each developer's machine, distributed via the team wiki

~/.claude/skills/ is user-scoped and not version-controlled. Distributing via wiki requires manual copying and creates drift between developers' versions. The requirement is automatic availability on clone.

B In the repository's .claude/skills/ directory, committed to version control

.claude/skills/ is project-scoped and version-controlled. When any team member clones the repository, the skill is automatically available as a /extract-service command. Updates to the skill flow through normal git pull, ensuring all developers have the same version.

D In a shared MCP server that all team members connect to

MCP servers provide tool integrations and external data access, not reusable prompt-based workflows. A skill file is the correct mechanism for a guided multi-step procedure.

WHERE THIS COMES FROM

Lesson 3.2: Custom Slash Commands and Skills (Project skill scoping)

Claude Code: Skills ↗

Q52 · D2 2.1 TOOL-SCHEMA-DESIGN / DESCRIPTIONS q-2-1-011 · Enterprise Data Platform with Federated Queries
INCORRECT

Your federated query platform exposes several similar search tools, and Claude keeps routing requests to the wrong one. Which changes improve tool selection reliability? (Select 3)

E Merge the similar tools into one generic tool with a mode parameter.

The guide moves in the opposite direction: split generic tools into purpose-specific tools with defined contracts.

C Check the system prompt for keyword-sensitive instructions that create unintended associations with particular tools.

System prompt wording can override well-written descriptions, so it belongs in the same review. (correct answer)

B Trim every description to one short sentence so the model relies on tool names alone.

Minimal descriptions cause unreliable selection among similar tools; names alone cannot carry boundary information.

A Rename tools whose names overlap so each name reflects a distinct function.

The guide's example renames analyze_content to extract_web_results, removing the functional overlap that caused misrouting. (correct answer)

D Rewrite each description to state the tool's purpose, inputs and outputs, and when to prefer it.

Descriptions are the primary signal Claude uses to choose tools; differentiating them is the first fix for misrouting.

WHERE THIS COMES FROM

Lesson 2.1: Tool Interface Design (Misrouting)

Lesson 2.1: Tool Interface Design (Dialect-specific descriptions)

Anthropic: Tool Use Documentation ↗

Q53 · D2 2.1 TOOL-SCHEMA-DESIGN / DESCRIPTIONS q-2-1-010 · Enterprise Data Platform with Federated Queries

INCORRECT

The data platform's query_snowflake tool has a description that reads: 'Queries Snowflake data warehouse. Accepts SQL.' The agent correctly uses the tool but frequently sends queries using PostgreSQL-specific syntax (e.g. string_agg, which Snowflake spells LISTAGG) that Snowflake rejects. What is the most effective fix?

C Add a system prompt instruction listing all Snowflake-specific SQL functions the agent should use.

A system prompt instruction is brittle and consumes context tokens for every turn, even when the Snowflake tool is not being used. The tool description is the correct place for tool-specific dialect guidance.

A Add a SQL syntax validation layer in front of the MCP tool that rejects non-Snowflake syntax before execution.

A validation layer adds infrastructure complexity and maintenance burden. The root cause is that the tool description does not specify which SQL dialect to use. Fixing the description is the proportionate first step. (your answer)

B State in the tool description that it expects the Snowflake SQL dialect, not PostgreSQL.

The description says 'Accepts SQL' without naming the dialect, so the agent defaults to PostgreSQL syntax. Naming the Snowflake dialect and giving examples in the description (LISTAGG rather than string_agg, ILIKE is supported) resolves the misrouting at source. (correct answer)

D Have the MCP server automatically translate PostgreSQL syntax to Snowflake syntax before executing the query.

Auto-translation is fragile and cannot handle all dialect differences. It masks the problem rather than fixing it and introduces a complex transformation layer prone to edge cases.

WHERE THIS COMES FROM

Lesson 2.1: Tool Interface Design (Dialect-specific descriptions)

Q54 · D1 1.3 SUBAGENT-INVOCATION-CONTEXT / GOAL-ORIENTED-PROMPTS q-1-3-018 · Large-Scale Codebase Refactoring with Multi-Agent Claude Code

CORRECT

A coordinator delegates a large refactoring subtask to a specialist subagent, and the architect must write the subagent's prompt. Which prompt design best follows the guide's guidance on how a coordinator should instruct a subagent?

C Provide only the high-level goal with no success criteria, so the subagent has maximum freedom

A goal with no quality criteria gives the subagent no way to know when it has actually succeeded. Effective goal-oriented prompts pair the objective with explicit success criteria.

B Give an exact step-by-step procedure (open file X, change line 12, run command Y, then edit file Z) so the subagent cannot deviate

A rigid procedure constrains the subagent and breaks the moment reality diverges from the script. The guide specifically warns against procedural instructions in favour of goal-oriented ones.

D Omit the goal and instead list the tools the subagent may use, letting it infer the objective from the toolset

Listing tools without stating the goal leaves the objective ambiguous, and the subagent may optimise for the wrong outcome. The prompt must state the goal.

A State the goal and its quality criteria (no breaking API changes, all tests pass) and let the subagent choose how to get there.

Goal-oriented prompts specify what to achieve and the quality bar, not a fixed procedure. Pairing the objective with success criteria lets the subagent adapt its approach when it meets something unexpected, which is what the guide recommends.

WHERE THIS COMES FROM

Lesson 1.3: Subagent Invocation and Context Passing (Goal-oriented prompts)

Q55 · D2 2.4 MCP-SERVER-INTEGRATION / RESOURCES q-2-4-012 · Enterprise Data Platform with Federated Queries

CORRECT

A data-analysis agent connects to an MCP server that exposes 40 database tables. Before almost every query, the agent makes several exploratory tool calls to discover which tables exist and what columns they hold, adding latency and token cost to each task. The server author wants to remove this discovery overhead. What is the most effective change?

D Reduce the server to the five most frequently queried tables so there is less to discover.

This discards capability the agent legitimately needs and still leaves discovery calls for the remaining tables. It treats the symptom, not the discovery mechanism.

C Enlarge the agent's context window so it can retain the results of the discovery calls across turns.

A larger context window does not stop the agent making the discovery calls; it still pays the latency and token cost on each task and merely stores the results.

A Expose the table catalogue and column schemas as MCP resources for upfront visibility.

MCP resources expose content catalogues such as database schemas, documentation hierarchies, and issue summaries, giving the agent visibility into available data upfront. The agent no longer needs to call list_tables and then describe_table for each table, which removes the discovery overhead entirely.

B Add a describe_schema tool and instruct the agent in its system prompt to call it before every query.

This keeps an exploratory tool call on every task, which is the exact overhead the author wants to remove. Resources present the catalogue without a per-task call.

WHERE THIS COMES FROM

Lesson 2.4: MCP Server Integration (MCP resources)

Model Context Protocol: Resources ↗

Q56 · D3 3.4 PLAN-MODE-EXECUTION / PLAN-MODE q-3-4-006 · Large-Scale Codebase Refactoring with Multi-Agent Claude Code

CORRECT

A developer extracting the notification subsystem from a monolith faces 12 cross-module dependencies, 3 messaging patterns, and several valid extraction strategies. They start in direct execution mode. After moving 8 files, the chosen approach breaks circular dependencies with the user-profile module. What should they have done differently?

C Used direct execution but processed only 2 files at a time to catch problems earlier

Smaller batches in direct execution do not address the fundamental issue: the developer chose a strategy without understanding the full dependency landscape. The circular dependency would still be discovered mid-migration, just

slightly sooner.

D Delegated the entire extraction to a subagent to isolate the risk

Delegating to a subagent does not change the outcome if the subagent also uses direct execution without exploring dependencies. The mode of execution (plan vs direct) matters more than who executes it.

A Used direct execution but with more detailed upfront instructions specifying how to handle each dependency

The developer did not know about the circular dependency with the user-profile module upfront. More detailed instructions cannot account for undiscovered dependencies. The codebase needed exploration first.

B Used plan mode to map the 12 dependencies and evaluate extraction strategies before committing

A subsystem with 12 dependencies, multiple messaging patterns, and several valid strategies is a textbook plan mode scenario. Plan mode would have mapped the dependency graph and revealed the circular dependency with user-profile before any files were moved, avoiding costly rework.

WHERE THIS COMES FROM

Lesson 3.4: Plan Mode vs Direct Execution (Plan mode for migrations)

Lesson 3.4: Plan Mode vs Direct Execution (Recognising complexity)

Q57 · D4 4.1 SYSTEM-PROMPTS / KEYWORD-OVERLAP q-4-1-005 · CI/CD Pipeline Integration CORRECT

A CI/CD system prompt defines two review categories with the instructions 'Check for security vulnerabilities in each function' and 'Check for performance issues in each loop'. The model frequently calls `performance_check` for security issues found inside loops, and `security_check` for performance issues in security-sensitive functions. What is the root cause and best fix?

D Force tool_choice to 'auto' and let the model determine the correct tool based on the content of each finding

tool_choice 'auto' is likely already the setting. The problem is that the model's tool selection is being misled by keyword overlap in the prompt, not by the tool_choice configuration.

A The model is confused because loops can have both security and performance issues; add a rule that security always takes priority over performance

Priority rules add complexity without fixing the root cause. The model is confusing tools because the instruction keywords overlap with both tool names and each other.

B Keyword overlap between the instruction phrasing and the tool names causes the confusion; rewrite them to use distinct, non-overlapping terms.

When instruction text like 'check for security in each function' overlaps with tool name patterns, the model conflates instruction keywords with tool selection cues. Using distinct terminology for instructions versus tool names eliminates this cross-contamination.

C Add more detailed tool descriptions explaining exactly when each tool should be called

More detailed descriptions help but do not fix the root cause if the instruction text still contains overlapping keywords that cue the wrong tool.

WHERE THIS COMES FROM

Lesson 4.1: System Prompts with Explicit Criteria (Keyword overlap)

Anthropic: Tool Use Documentation ↗

Q58 · D4 4.5 BATCH-PROCESSING / PROMPT-CACHING q-4-5-006 · CI/CD Pipeline Integration CORRECT

Your Claude Code setup uses a complex system prompt with detailed review instructions, code examples, and few-shot demonstrations that total 4,000 tokens. Each code review request adds 500-2,000 tokens of code to review. The team wants to optimise costs since the same review instructions are used for every request. What is the most effective optimisation?

D Reduce the number of few-shot examples from 4 to 1 and lower the temperature to compensate for the reduced guidance

Lower temperature does not compensate for reduced examples. Fewer examples degrade quality. Prompt caching preserves the full prompt while reducing costs.

A Shorten the system prompt by removing few-shot examples to reduce per-request token costs

Removing few-shot examples degrades review quality. The 4,000-token static prompt is the ideal candidate for caching, not trimming.

B Enable prompt caching with the static review instructions and examples first and the code-to-review at the end.

Prompt caching caches the static prefix and reuses it across requests. With 4,000 tokens of static content reused for every review, caching provides significant cost savings without sacrificing review quality. Dynamic code goes at the end so the prefix remains cacheable.

C Switch all code reviews to the Message Batches API for 50% cost savings

Code reviews in Claude Code are interactive, blocking workflows. The Batches API's 24-hour window makes it unsuitable for developer-facing reviews.

WHERE THIS COMES FROM

Lesson 4.5: Batch Processing and Prompt Optimisation (Prompt caching layout)

Anthropic: Prompt Caching ↗

Q59 · D1 1.2 ORCHESTRATION-PATTERNS / DELEGATION q-1-2-011 · Large-Scale Codebase Refactoring with Multi-Agent Claude Code INCORRECT

The coordinator receives two requests simultaneously: (1) rename a utility class used across 60 files, and (2) add a new health-check endpoint to one microservice. A developer proposes delegating both tasks to subagents. What is the optimal delegation strategy?

A Delegate both to subagents — task 1 to a rename specialist and task 2 to an API specialist

While task 1 requires delegation due to its scope, task 2 is a simple, well-scoped addition to a single file. Delegating trivial tasks to subagents adds unnecessary overhead from context injection and result collection. (your answer)

D Delegate task 2 to a subagent and have the coordinator handle task 1, since the coordinator has full codebase awareness needed for the rename

This inverts the correct strategy. The coordinator's full codebase awareness does not prevent attention dilution across 60 files. The rename needs a subagent with focused context; the simple health check should stay with the coordinator.

C Delegate task 1 (60-file rename) to a subagent with scoped context, and have the coordinator handle task 2 (single-file health check) directly

The coordinator should delegate when tasks are large or require specialist focus, and handle tasks directly when they are simple and well-scoped. A 60-file rename benefits from a dedicated subagent with focused context. A single-file endpoint addition is trivial enough for the coordinator to handle without delegation overhead. (correct answer)

B Handle both tasks directly in the coordinator to minimise subagent communication overhead

Task 1 affects 60 files. Processing that many files directly in the coordinator will cause attention dilution. The coordinator should delegate large, multi-file tasks and only handle simple ones directly.

WHERE THIS COMES FROM

Lesson 1.2: Multi-Agent Orchestration (Coordinator responsibilities)

Lesson 1.6: Task Decomposition Strategies (Attention dilution)

Q60 · D4 4.2 FEW-SHOT-PROMPTING / FEW-SHOT-STYLE q-4-2-002 · CI/CD Pipeline Integration CORRECT

Your Claude Code agent generates unit tests in the CI pipeline. When given detailed instructions alone, it produces tests with inconsistent assertion styles: sometimes using expect().toBe(), sometimes assert.equal(), and occasionally mixing both in the same file. Adding more detailed instructions about assertion style did not fix the problem. What should you do next?

B Add a linter post-processing step to automatically convert all assertions to a single style

Post-processing masks the problem rather than solving it. The model should learn to produce consistent output directly, which few-shot examples achieve more effectively.

D Add 2-4 few-shot examples demonstrating the desired assertion style with reasoning for each testing decision, covering edge cases like async functions and error handling

Few-shot examples are the most effective technique when detailed instructions alone produce inconsistent formatting. Including 2-4 examples with reasoning teaches generalisation to novel patterns, not just pattern-matching. Covering varied scenarios (async, error handling) ensures the model generalises correctly.

A Switch to a different model that better follows formatting instructions

The issue is not model capability. When detailed instructions alone produce inconsistent formatting, the correct intervention is few-shot examples, not a model change.

C Add 2-4 few-shot examples showing complete test files with the desired assertion style and reasoning for why that style was chosen over alternatives

This is partially correct but incomplete. The examples only cover the assertion style preference. Without covering varied scenarios (async functions, error handling), the model pattern-matches the specific cases shown rather than generalising the style consistently across all test types.

WHERE THIS COMES FROM

Lesson 4.2: Few-Shot Prompting (Few-shot examples)

Anthropic: Multishot (Few-Shot) Prompting ↗